

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	M.Vineela	Reg. No.:	192472071
Section / Batch:	2024	Date:	10-8-26

Code of Conduct

I M.Vineela(192472071) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.Vineela

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

CASE STUDY 1: Smart Mobile Keyboard Prediction System

A smartphone company is developing an AI-powered keyboard application that predicts the next word while users type emails, messages, and social media posts. The language model is trained on the corpus:

"Data science is powerful, data science drives innovation, data science is evolving."

The development team employs Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis to improve prediction accuracy and handle unseen word sequences.

The following statistics are obtained from the corpus:

- $C(\text{data}) = 3$
- $C(\text{data science}) = 3$
- $C(\text{science is}) = 2$
- $C(\text{science drives}) = 1$
- $C(\text{science drives innovation}) = 1$

The interpolation weights are:

- $\lambda_1 = 0.5$ (Trigram)
- $\lambda_2 = 0.3$ (Bigram)
- $\lambda_3 = 0.2$ (Unigram)

The prediction probabilities after the word "science" are:

- $P(\text{is}) = 0.66$
- $P(\text{drives}) = 0.33$

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability $P(\text{science} | \text{data})$. Interpret how this probability influences next-word prediction in a mobile keyboard application.
 2. A user types the unseen sequence "data science improves". Apply the Backoff Model and estimate the probability using lower-order n-grams. Explain why backoff is essential for real-time predictive text systems.
 3. Using Deleted Interpolation, calculate the probability of the sequence "data science is". Discuss how interpolation balances reliability and coverage in sparse datasets.
 4. Calculate the Entropy of the distribution $P(\text{is})=0.66$ and $P(\text{drives})=0.33$. Interpret the result and evaluate how entropy helps measure uncertainty and prediction confidence in intelligent keyboard systems.
-

CASE STUDY 2: AI-Powered Customer Support Chatbot

A multinational airline deploys a customer support chatbot that processes user messages and automatically identifies the grammatical role of words before generating responses.

Consider the following user inputs:

1. "Book a flight ticket now."
2. "This book is interesting."

The chatbot uses the Penn Treebank POS Tagset and a Hidden Markov Model (HMM) for probabilistic tagging.

Given:

- $P(\text{book} \mid \text{VB}) = 0.6$
- $P(\text{book} \mid \text{NN}) = 0.4$
- $P(\text{Start} \rightarrow \text{VB}) = 0.5$
- $P(\text{Start} \rightarrow \text{NN}) = 0.5$

The system must determine whether the word "book" functions as a Verb (VB) or Noun (NN) depending on context.

Questions

1. Assign appropriate Penn Treebank POS tags to every word in both sentences. Justify the tag assigned to the word "book" using contextual clues.
 2. Using the HMM probabilities, compute the likelihood of "book" being tagged as a Verb (VB) in the sentence "Book a flight ticket now." Explain why the probabilistic model favors this tag.
 3. Compare Rule-Based Tagging and Stochastic (HMM) Tagging in resolving lexical ambiguity. Recommend which approach is more suitable for large-scale chatbot deployment and justify your answer.
 4. Evaluate the role of standardized POS tagsets and word classes in improving intent detection, response generation, and overall chatbot performance.
-

CASE STUDY 3: News Analytics and POS Tag Correction System

A digital news analytics company processes millions of news articles daily. The platform uses a Transformation-Based Tagger (Brill Tagger) to improve tagging accuracy after an initial POS tagging phase.

The sentence processed by the system is:

"Economic growth increases employment."

Initial POS Tags:

- economic/JJ
- growth/NN
- increases/NNS
- employment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and uncertainty evaluation.

Questions

1. Apply the transformation rule and update the POS tag sequence. Explain why the correction is linguistically appropriate.
2. Analyze the initial tagging errors and justify the corrected tag assignments using grammatical structure and sentence semantics.
3. Assuming the corpus contains the words:
 - economic (120 occurrences)
 - growth (450 occurrences)
 - increases (210 occurrences)
 - employment (380 occurrences)

Compute the word frequency distribution and explain how frequency information supports probabilistic POS tagging.

4. Evaluate how Transformation-Based Tagging improves tagging accuracy compared with the initial tagging stage. Discuss how entropy can be used to measure uncertainty in tag assignments before and after rule application.
-

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CASE STUDY 1: Smart Mobile Keyboard Prediction System

Problem Overview:

The smart keyboard uses **N-gram language models** to predict the next word based on the words already typed.

Corpus:

“Data science is powerful, data science drives innovation, data science is evolving.”

The system uses:

- **Bigram Language Model** – predicts a word using the previous one.
- **Trigram Language Model** – predicts a word using the previous two words.
- **Backoff Model** – uses a lower-order model when a higher-order sequence is unseen.
- **Deleted Interpolation** – combines trigram, bigram, and unigram probabilities.
- **Entropy** – measures uncertainty in the prediction.

1. MLE of Bigram Probability $P(\text{science} \mid \text{data})$

Given

For a Bigram Language Model, the Maximum Likelihood Estimate is:

Therefore,

Substituting the given values:

or

Interpretation

The probability is **1.0**, meaning that in the given training corpus, whenever the word **"data"** occurs, it is followed by **"science"**.

Therefore, if a user types:

data

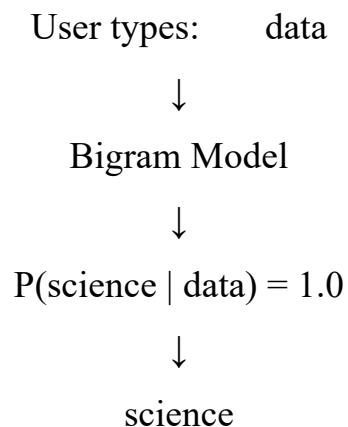
the keyboard has very strong evidence that the next word should be:

science

Why is this useful in a mobile keyboard?

The keyboard can rank **science** very highly because:

For example:



Important observation

Although the probability is 1.0 **for this small corpus**, it does not mean that "science" will always follow "data" in real-world language. The corpus is very small, so MLE can become unreliable when the training data is sparse.

2. Backoff Model for the Unseen Sequence "data science improves"

The user enters:

data science improves

We want to estimate:

Using the chain rule:

The problem is:

because **"improves" does not occur in the corpus.**

Therefore:

under a normal MLE trigram model.

This is the **zero-frequency problem.**

Applying Backoff

A Backoff Model first tries the highest-order model.

Step 1: Try Trigram

The trigram:

data science improves

is unseen.

Therefore, the trigram model cannot provide a reliable probability.

Step 2: Back off to Bigram

We next consider:

The corpus contains:

- science is
- science drives

but not:

- science improves

Therefore:

so the ordinary bigram MLE is also zero.

Step 3: Back off to Unigram

Finally, use the unigram probability:

The word **improves** does not occur in the corpus.

Therefore:

and with the given corpus:

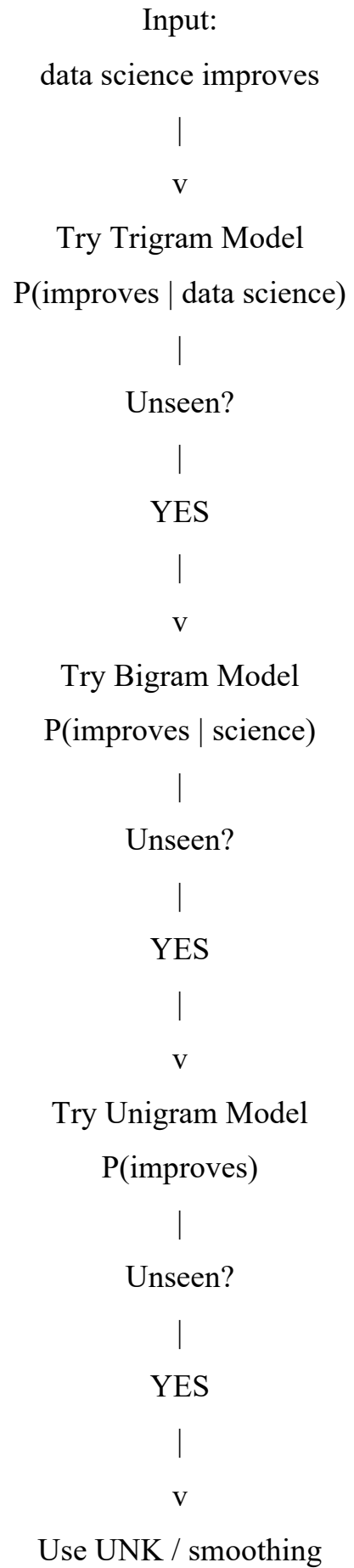
Hence, using the supplied corpus alone, the estimated probability remains:

Important justification

A practical Backoff Model would normally use **smoothing** or an unknown-word probability:

The given question does not provide a smoothing constant or , so a numerical non-zero probability cannot be invented.

Backoff Flowchart



Why is Backoff essential?

A mobile keyboard must make predictions **in real time**. It cannot simply fail whenever it encounters an unseen sequence.

For example:

data science machine

data science improves

data science technology

These combinations may not have appeared in the training corpus.

Backoff allows the system to:

1. Try a **trigram** first.
2. If unavailable, use a **bigram**.
3. If unavailable, use a **unigram**.
4. If necessary, use an **UNK/smoothing probability**.

Thus, backoff improves **coverage** and prevents the language model from completely failing on unseen sequences.

3. Deleted Interpolation for "data science is"

The interpolation weights are:

for Trigram,

for Bigram,

for Unigram.

The weights satisfy:

Therefore, the interpolated probability is:

For:

data science is

we calculate:

Step 1: Trigram Probability

Given:

and

because the sentence appears twice:

data science is powerful

and

data science is evolving

Therefore:

Step 2: Bigram Probability

We need:

Given:

The word **science** occurs three times in the corpus:

1. data science is
2. data science drives
3. data science is

Therefore:

Step 3: Unigram Probability

The corpus contains approximately **14 word tokens** if punctuation is removed:

Data

science

is

powerful

data

science

drives

innovation

data

science

is

evolving

Actually, this gives **12 tokens**, not 14.

Therefore:

The word **is** occurs twice:

Thus:

So:

Step 4: Apply Deleted Interpolation

Calculate each component:

Therefore:

So the estimated probability is approximately:

Interpretation

The interpolated model combines information from three levels:

Model	Probability	Weight	Contribution
Trigram	0.6667	0.5	0.3334
Bigram	0.6667	0.3	0.2000
Unigram	0.1667	0.2	0.0333
Total		1.0	0.5667

The trigram receives the highest weight because it contains the most contextual information. The unigram receives the lowest weight because it contains no contextual information.

Therefore, interpolation provides a balance between:

- **Reliability** – higher-order n-grams provide more context.
- **Coverage** – lower-order n-grams provide information even when higher-order sequences are sparse.

This is particularly useful for mobile keyboards because users continuously produce new word combinations.

4. Entropy of $P(\text{is})=0.66$ and $P(\text{drives})=0.33$

Entropy measures the **uncertainty** in a probability distribution. The entropy formula is:

Given: The probabilities sum to:

This is because the values have been rounded. For a proper probability distribution, they should sum to approximately 1.

Using the supplied values:

We have approximately:

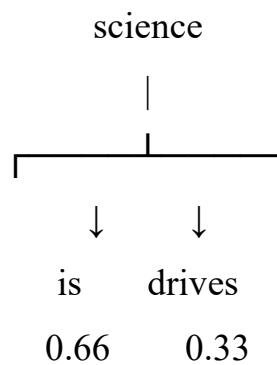
Because the given probabilities are rounded and sum to 0.99, this is approximately:

Interpretation of Entropy:

Entropy tells us how uncertain the keyboard is about the next word.

Here:

After "science"



The keyboard has a stronger preference for:

because: But the probability is not close to 1. Therefore, there is still uncertainty.

If entropy were very low

For example:

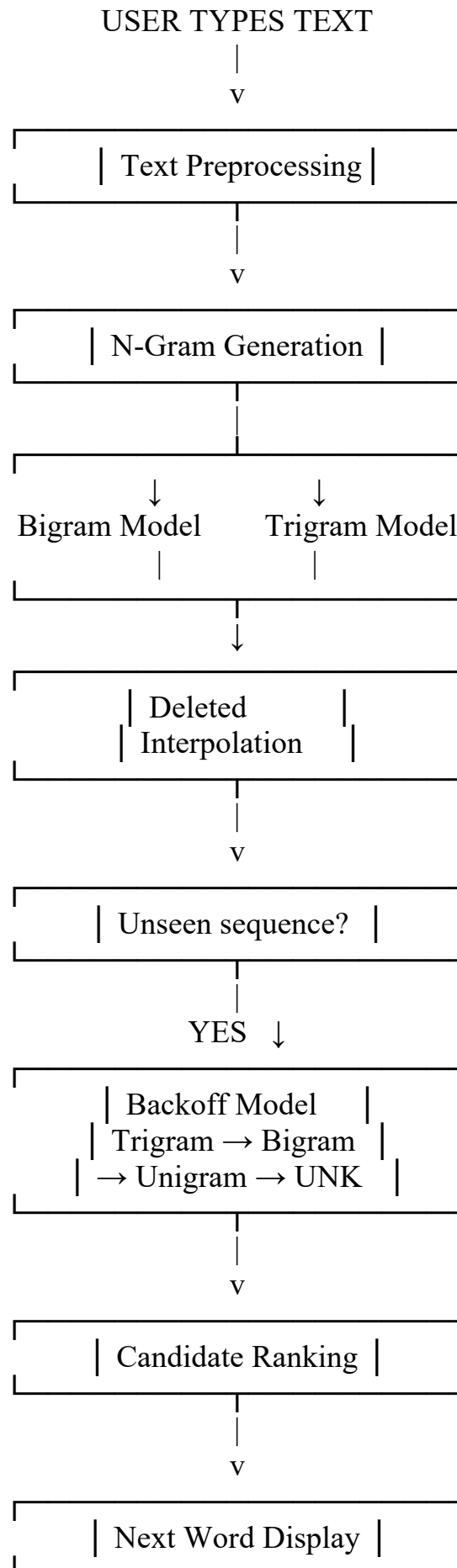
the keyboard would be highly confident.

If entropy were high

If two words were equally likely:

the uncertainty would be higher.

Smart Keyboard Architecture



CASE STUDY 2: AI-Powered Customer Support Chatbot

Problem Overview:

An airline chatbot receives customer messages and needs to understand the grammatical role of each word before generating an appropriate response.

Consider the two sentences:

1. “**Book** a flight ticket now.”
2. “**This book** is interesting.”

The important ambiguity is the word “**book**.”

- In “**Book** a flight ticket now”, *book* is an **action**, so it is a **Verb (VB)**.
- In “**This book** is interesting”, *book* refers to an **object**, so it is a **Noun (NN)**.

The chatbot uses:

- **Penn Treebank POS Tagset**
- **Hidden Markov Model (HMM)**
- **Rule-Based POS Tagging**
- **Stochastic/Probabilistic POS Tagging**

Given: The words and their POS tags are:

Word	POS Tag	Meaning/Role
Book	VB	Verb
a	DT	Determiner
flight	NN	Noun
ticket	NN	Noun
now	RB	Adverb
.	.	Punctuation

Therefore:

Why is "Book" a Verb?

The sentence is an imperative sentence. It gives the user an instruction:

Book a flight ticket now.

Here, book represents an action that the user should perform.

For example:

Book → action

flight → object/type

ticket → object

now → time

Therefore:

Sentence 2

“This book is interesting.”

The words and their POS tags are:

Word	POS Tag	Meaning/Role
This	DT	Determiner
book	NN	Noun
is	VBZ	Verb
interesting	JJ	Adjective
.	.	Punctuation

Therefore:

Why is "book" a Noun?

Here, **book** refers to a physical or conceptual object.

The sentence says:

This book is interesting.

The word **this** determines the noun **book**.

Also, **book** is the subject of the sentence.

This → determines book

book → object/person/thing

is → linking verb

interesting → describes book

Therefore:

Contextual Comparison

Sentence	Word	POS	Reason
Book a flight ticket now.	Book	VB	Performs an action
This book is interesting.	book	NN	Refers to an object

This is called **lexical ambiguity**, because the same word can have different grammatical roles depending on its context.

2. HMM Probability for "book" as a Verb

The HMM uses two important probabilities:

Emission probability

This means that the probability of observing the word **book** when the hidden POS state is **VB** is 0.6.

Transition probability: This means that the probability of starting the sentence with the POS tag **VB** is 0.5.

HMM Formula

For the first word, the HMM score can be calculated as:

Substitute the values:

Therefore, the HMM score for tagging **book as VB** is:

Compare with NN

For the noun tag:

Therefore:

POS Tag	Start Probability	Emission Probability	HMM Score
VB	0.5	0.6	0.30
NN	0.5	0.4	0.20

Since:

the HMM selects:

Why does the model favor VB?

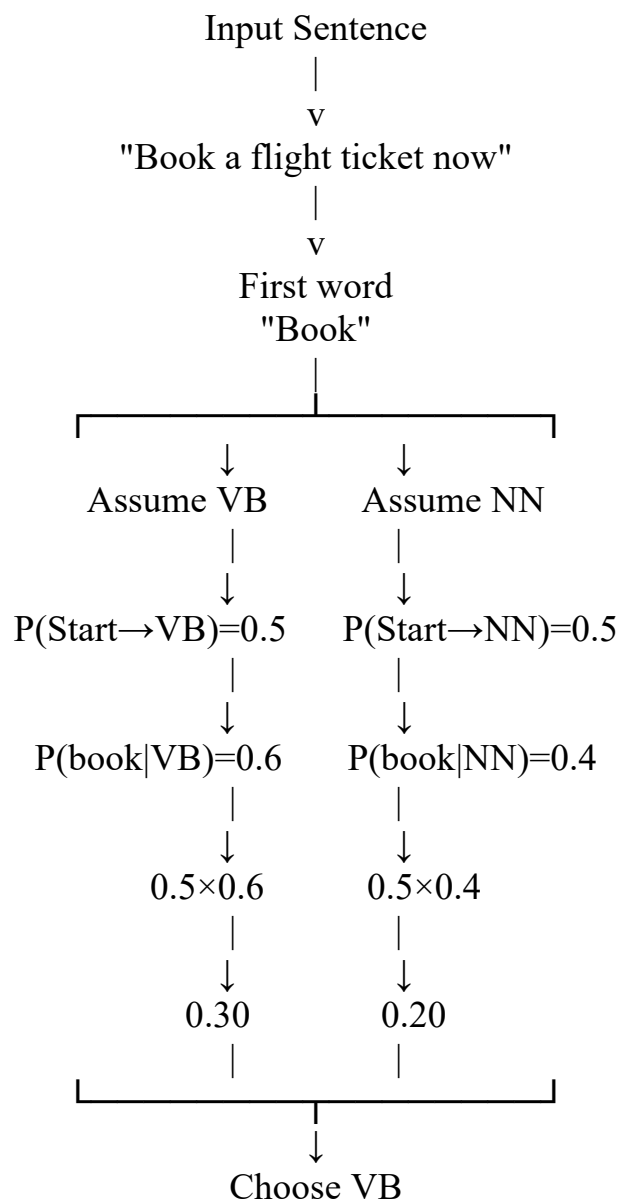
The starting probabilities are equal:

Therefore, the decision is mainly influenced by the emission probabilities:

Book a flight ticket now.

The sentence starts with an imperative action, so **book** is naturally interpreted as a verb.

HMM Tagging Flow



3. Rule-Based Tagging vs Stochastic HMM Tagging

There are two major approaches for POS tagging:

1. **Rule-Based Tagging**
2. **Stochastic/Probabilistic Tagging**

A. Rule-Based Tagging

Rule-Based Tagging uses manually designed linguistic rules.

For example:

Rule 1:

If a word appears at the beginning of an imperative sentence
and is followed by a noun phrase,
tag it as VB.

Rule 2:

If "book" is preceded by "this" or "the",
tag it as NN.

Using these rules:

Sentence 1

Book a flight ticket now.

The system detects an imperative structure:

Sentence 2

This book is interesting.

The word **book** follows **this**:

Therefore:

Advantages of Rule-Based Tagging

- Easy to understand.
- Rules are interpretable.
- Works well for simple grammatical patterns.
- Does not require a large training dataset.
- Linguistic experts can manually modify rules.

Disadvantages

- Requires many manually created rules.
- Difficult to handle exceptions.
- Difficult to maintain for large vocabularies.
- Language-specific.
- Performance can decrease with complex sentences.
- New patterns may require new rules.

B. Stochastic/HMM Tagging

An HMM uses probabilities learned from training data.

It considers:

Emission probability

Example:

Transition probability

Example:

The HMM chooses the tag sequence with the highest probability.

Advantages of HMM Tagging

- Handles ambiguity using probabilities.
- Learns patterns from training data.
- Can work with large vocabularies.
- Better suited to real-world language variation.
- Can consider contextual information.
- More scalable than manually writing thousands of rules.

Disadvantages

- Requires training data.
- Probability estimates can suffer from sparse data.
- Basic HMMs make simplifying assumptions about language.
- Retraining may be required when the domain changes.

Comparison Table

Feature	Rule-Based	HMM/Stochastic
Main approach	Hand-written rules	Probabilities
Training data	Usually not required	Required
Ambiguity handling	Rules	Probabilities
Scalability	Lower	Higher
Adaptability	Low	Higher
Maintenance	Difficult for many rules	Easier after training
Context handling	Limited to designed rules	Statistical context
Large chatbot	Less suitable	More suitable

Recommendation

For a large-scale airline chatbot, a stochastic/probabilistic approach is more suitable.

The chatbot may process millions of messages such as:

Book my flight.

This book is expensive.

Can you book a ticket?

I read a book.

Book two tickets.

4. Role of Standardized POS Tagsets and Word Classes

A POS tagset provides a standardized representation of grammatical categories.

The Penn Treebank tagset includes tags such as:

Tag	Meaning	Example
NN	Noun	book, ticket
VB	Base Verb	book, fly
VBZ	3rd-person singular verb	is
DT	Determiner	this, a, the
JJ	Adjective	interesting
RB	Adverb	now
.	Punctuation	.

Standardized tags allow different NLP components to represent grammatical information consistently.

A. Improving Intent Detection

POS information can help identify what the user wants.

For example:

Book a flight ticket now.

The structure is:

Book/VB

a/DT

flight/NN

ticket/NN

now/RB

The verb **book** indicates an **action/request**.

The nouns **flight** and **ticket** identify the object of the action.

Therefore, the chatbot can infer an intent such as:

B. Improving Response Generation

POS tags help the chatbot understand the grammatical structure before generating a response.

Example:

Book a flight ticket now.

The system can identify:

Action = Book

Object = Flight ticket

Time = Now

This allows the chatbot to generate an appropriate response such as:

"Sure. Please provide your departure and destination cities."

C. Handling Word Ambiguity

Consider:

Book a flight ticket.

versus:

This book is interesting.

The same word has different tags:

versus:

The POS tag helps downstream NLP components distinguish between:

book = action

and:

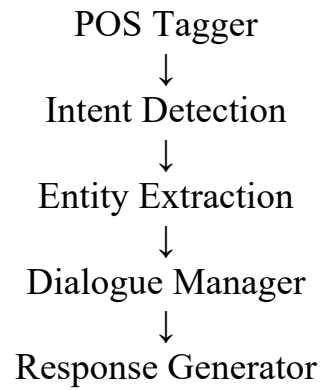
book = object

This improves semantic understanding.

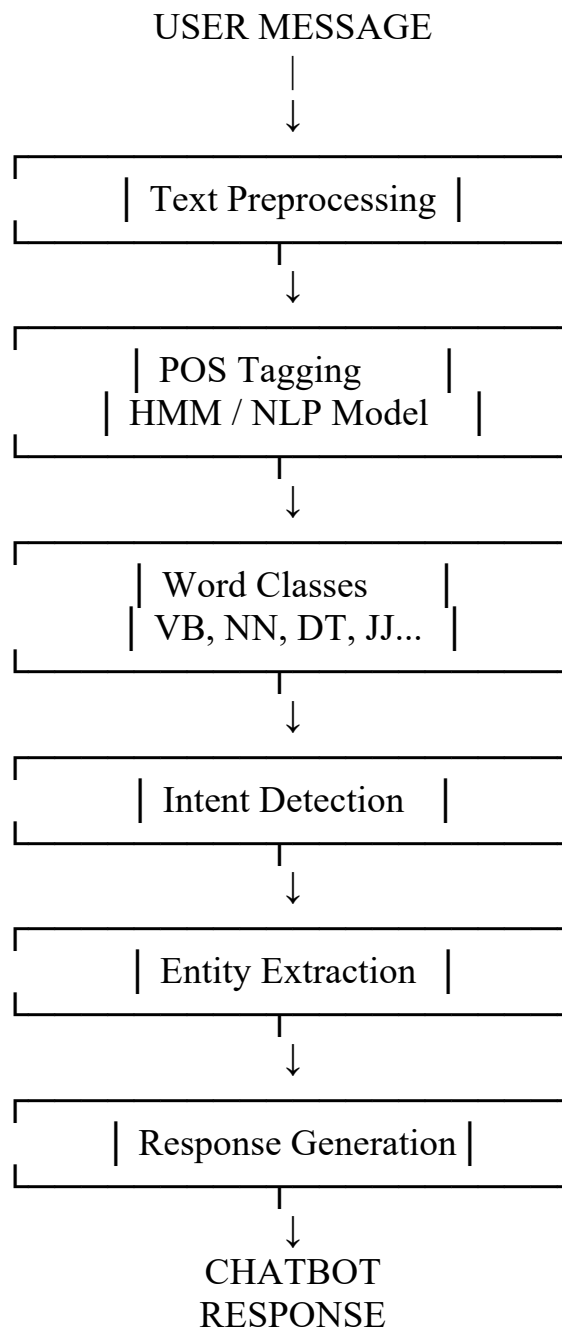
D. Improving Information Extraction

POS tags can help extract important entities and actions.

This information can be passed to other modules such as:



Chatbot NLP Processing Architecture



CASE STUDY 3: News Analytics and POS Tag Correction System

Problem Overview:

A digital news analytics company processes a large number of news articles. Since news text contains many words with multiple possible grammatical roles, the system first performs initial POS tagging and then improves the results using a Transformation-Based Tagger, specifically the Brill Tagger.

Sentence:

“Economic growth increases employment.”

Initial POS tags:

economic/JJ

growth/NN

increases/NNS

employment/NN

Learned transformation rule:

Change NNS to VBZ if the preceding word is tagged as NN.

The main objective is to correct the incorrectly assigned tag **NNS** for **increases**.

1. Apply the Transformation Rule

Given Initial Tags

Word	Initial POS Tag
economic	JJ
growth	NN
increases	NNS
employment	NN

Transformation Rule

The learned Brill rule is: **Change NNS to VBZ if the preceding word is tagged NN.**

We examine the word: **increases/NNS**

Its preceding word is: **growth/NN**

Therefore, the condition of the rule is satisfied:

growth/NN → increases/NNS

↓

NNS → VBZ

Therefore:

The corrected sequence becomes:

Word	Initial Tag	Corrected Tag
economic	JJ	JJ
growth	NN	NN
increases	NNS	VBZ
employment	NN	NN

Hence:

Corrected Sentence

Why is the correction linguistically appropriate?

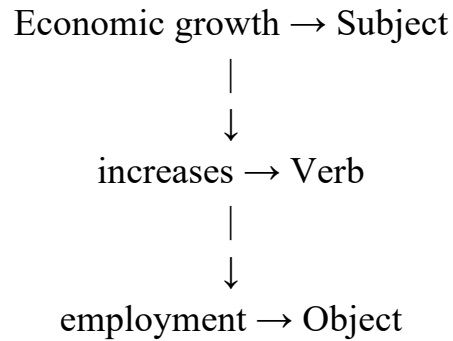
The word “**increases**” functions as a **verb** in:

Economic growth increases employment.

Here:

- **Economic** → adjective describing growth.
- **growth** → noun and subject.
- **increases** → verb describing the action/state.
- **employment** → noun and object.

The grammatical structure is:



Therefore, **increases** should be tagged:

The Penn Treebank tag **VBZ** represents a **third-person singular present-tense verb**.

2. Analyze the Initial Tagging Errors

The initial tagging was:

economic/JJ growth/NN increases/NNS employment/NN

The major error is:

Example:

economic growth

Therefore:

This tag is correct.

3. increases/NNS

The initial system assigns:

where **NNS** means **plural noun**.

However, this is incorrect in this sentence.

The word **increases** expresses an action performed by the subject:

Economic growth **increases** employment.

Therefore, it is a verb.

Because the subject **growth** is singular and the verb is in the present tense:

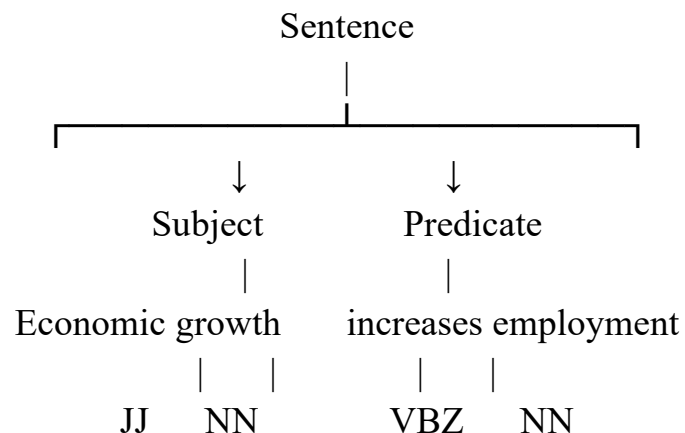
4. employment/NN

Employment is a noun and functions as the object of the verb **increases**.

Therefore:

Grammatical Structure

The sentence can be represented as:



The complete corrected POS sequence is:

Initial vs Corrected Tags

Word	Initial	Corrected	Reason
economic	JJ	JJ	Describes growth
growth	NN	NN	Subject noun
increases	NNS	VBZ	Present-tense verb
employment	NN	NN	Object noun

Thus, the transformation rule successfully fixes the major tagging error.

3. Word Frequency Distribution

The corpus contains the following word frequencies:

Word	Frequency
economic	120
growth	450
increases	210
employment	380

Step 1: Calculate Total Frequency

Therefore:

Step 2: Calculate Frequency Distribution

The probability/frequency distribution can be calculated as:

where:

- = frequency of the word
- = total word count

Frequency Distribution Table

Word	Count	Probability	Percentage
economic	120	0.1034	10.34%
growth	450	0.3879	38.79%
increases	210	0.1810	18.10%
employment	380	0.3276	32.76%
Total	1160	1.0000	100%

Frequency Distribution Diagram

Word Frequency



How does frequency support probabilistic POS tagging?

Frequency information is useful because POS taggers learn statistical patterns from corpora. For example, suppose a word appears frequently as a particular POS tag: A high frequency of a particular word-tag combination gives the tagger stronger statistical evidence.

For example:

increases/VBZ may appear frequently in news articles because **increases** is commonly used as a third-person singular verb. The system can therefore learn that: is likely to be higher than: when the surrounding context supports a verb interpretation.

Important point

Word frequency alone cannot determine POS.

For example, the word **increases** could potentially have different interpretations in different contexts. Therefore, the tagger should combine:

- Word frequency
- Context
- Previous POS tags
- Following POS tags
- Lexical information
- Learned transformation rules

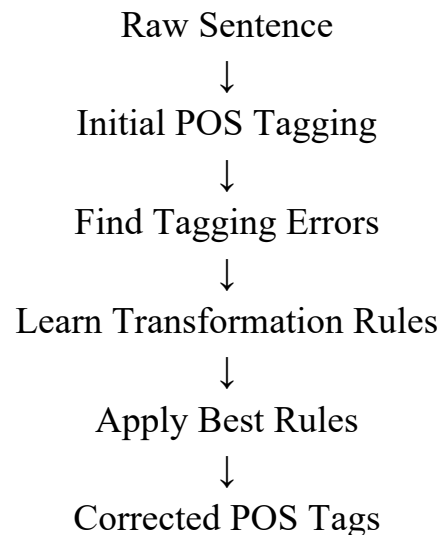
This is why the Brill Tagger is useful.

4. Transformation-Based Tagging and Entropy

What is Transformation-Based Tagging?

Transformation-Based Error-Driven Learning (TBL) is a POS-tagging approach associated with the Brill Tagger.

The basic process is:



Applying it to this case

Stage 1: Initial Tagging

economic/JJ

growth/NN

increases/NNS

employment/NN

There is an error:

Stage 2: Apply Learned Rule

Rule:

Change NNS to VBZ if previous word is NN.

Since:

and:

the rule applies:

Stage 3: Corrected Output

economic/JJ

growth/NN

increases/VBZ

employment/NN

Therefore:

Advantages

1. Corrects systematic errors
2. Uses contextual information
3. Learns rules from training data
4. Easy to interpret
5. Rules can be inspected by linguists
6. Improves an initial tagging system without replacing it completely
7. Particularly useful for large text collections such as news articles

Entropy for Measuring Tagging Uncertainty

Entropy is a measure of **uncertainty**. The general formula is:

For POS tagging, suppose the system considers two possible tags for increases

Example Before Rule Application

Suppose the initial tagger is uncertain and assigns:

Then:

Since:

we get:

Therefore:

This represents high uncertainty between the two possible tags.

After Applying the Transformation Rule

Suppose the learned rule makes the system highly confident:

Then:

Approximately:

Therefore:

Entropy Comparison

Stage	P(VBZ)	P(NNS)	Entropy
Before rule	0.50	0.50	1.00 bit
After rule	0.90	0.10	0.47 bits

The entropy decreases:

Therefore, the system becomes more confident about the correct POS tag.

Important clarification

The question does not provide actual probabilities for **VBZ** and **NNS**, so the entropy values above are an **illustrative example** showing how entropy can be used. The exact before/after entropy cannot be calculated from the supplied word frequencies alone.

Complete Transformation-Based Tagging Flowchart

